ArmForge Technical Report

Track 3 - Physical AI Challenge

Team: slobot

Application: ArmForge

Source code: https://github.com/alexis779/slobot-armforge

Overview

ArmForge is a ROCm-native pipeline for teaching an SO-ARM-101 manipulator to pick a cube and place it on a disk in Genesis. The stack centers on human key-action teleoperation, contact-stable simulation, LeRobot-shaped dataset export, and multi-environment throughput on AMD GPUs so the same teaching loop can feed reinforcement learning and real-robot workflows.

Why teleoperation matters

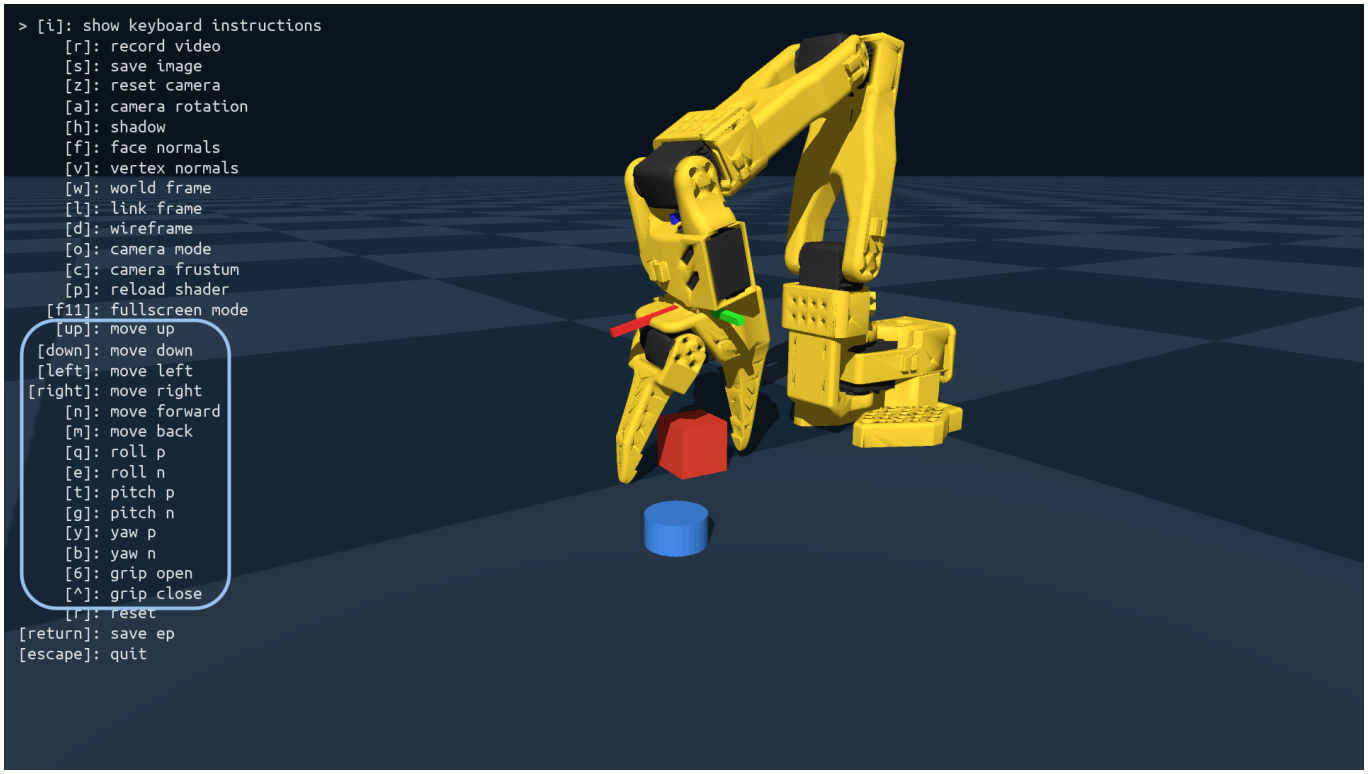
Teleoperation is paramount when the goal is to teach AI a contact-rich pick-and-place skill. Scripted motions and dense reward shaping alone often miss the timing, grasp commitment, and recovery behaviors that humans demonstrate under friction and partial observability. In ArmForge, an operator drives the SO-101 with keyboard key bindings that map to a 14-dimensional multi-hot action space (translations, roll/pitch/yaw nudges, and gripper open/close). Each simulation step records the held keys so idle and active frames are explicit. Those demonstrations become a LeRobot dataset: a full teleop episode and an idle-trimmed sibling that keeps only nonzero key-actions. The public release is available at https://huggingface.co/datasets/alexis779/so101_cube_disk, with the trimmed episode viewable in the LeRobot visualizer at https://huggingface.co/spaces/lerobot/visualize_dataset?path=%2Falexis779%2Fso101_cube_disk%2Fepisode_1. Teleop therefore supplies the curriculum that later RL or imitation methods can refine, rather than asking an untrained policy to invent grasp physics from scratch.

Why keyboard teleoperation

ArmForge uses keyboard teleoperation instead of a leader-follower arm pair so teaching stays accessible. A 2nd physical SO-101 (or other leader) is not required, nor is calibrated bimanual hardware, serial wiring, or a dedicated teleop station. Any machine that can run Genesis with a display and a keyboard can collect demos-on a laptop, an AMD Developer Cloud desktop session, or a ROCm workstation-without buying or shipping extra robot hardware. Discrete key bindings also match the multi-hot action representation used for replay and discrete RL (DQN), so the same interface that a human presses is the action space the policy can learn.

Cartesian control in the wrist frame

A second design choice is to command the arm in Cartesian end-effector space rather than joint space. Nudging individual joints is unnatural for pick-and-place: small joint deltas produce curved, coupled tip motion that is hard to aim at a cube or disk. Cartesian keys ("move forward", "up", "left") match how people think about reaching, while inverse kinematics maps those tip targets onto joint commands behind the scenes. Translations are applied in the current wrist / end-effector frame, not the world frame: "forward" means along the gripper's facing axis, and "up" stays aligned with the tool even after the wrist has been reoriented. That body-relative mapping keeps the same key meanings consistent as the grasp rotates, whereas world-frame axes would force the operator to remint mental coordinates after every roll or yaw. Genesis solves IK for the updated tip pose each step so the human stays in task space while the robot stays within joint limits.



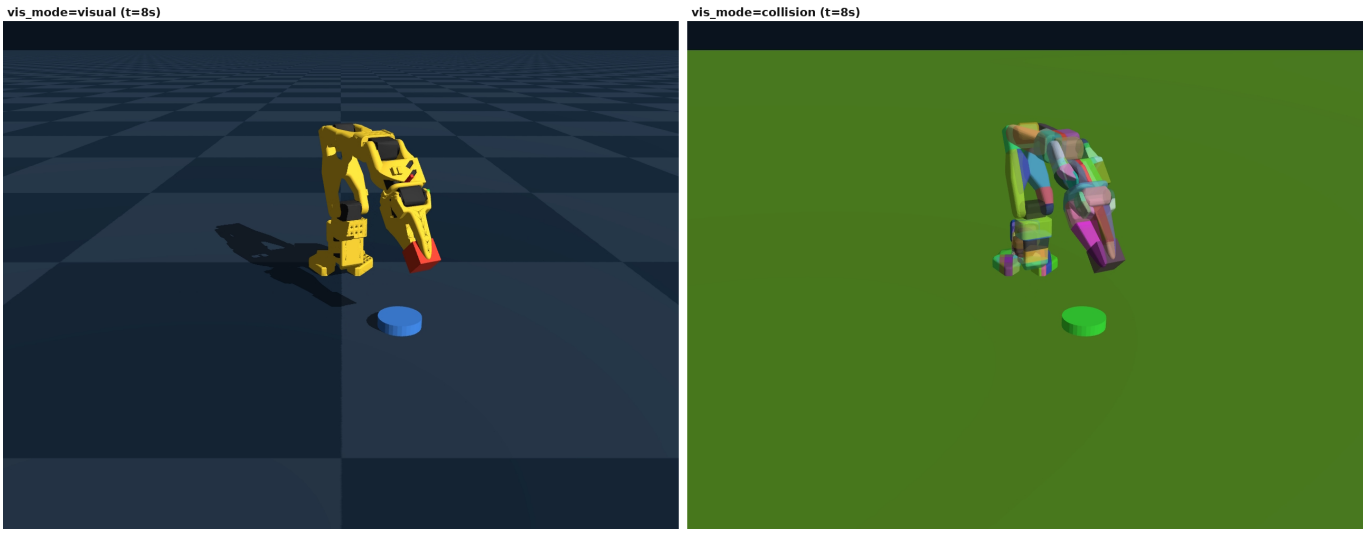
Genesis keyboard teleop of SO-101 cube-disk pick-and-place

Key bindings

Key	Action
Up	Move end-effector up (+Z in EE frame)
Down	Move end-effector down (-Z)
Left	Move end-effector left (-Y)
Right	Move end-effector right (+Y)
N	Move end-effector forward (-X)
M	Move end-effector back (+X)
Q	Roll +
E	Roll -
T	Pitch +
G	Pitch -
Y	Yaw +
B	Yaw -
6	Gripper open
^	Gripper close
R	Reset episode (discard buffer)
Enter	Save episode
Esc	Quit

Accurate collisions with CoACD

Accurate collision geometry is a prerequisite for that contact-rich teaching. A single convex hull per robot link is cheap but wrong for the SO-101's concave gripper and articulated links: hulls inflate volume, cause false contacts, and make pinching a cube unreliable. ArmForge enables convex decomposition through CoACD by setting `convexify=True` and forcing `decompose_robot_error_threshold=0.0` on the MJCF morph so every link is decomposed instead of remaining hull-only. The resulting collision meshes track the visual shape closely enough that jaw closing can capture the cube without phantom collisions from oversized hulls, which is essential for repeatable pick-and-place demos and for RL that depends on the same contact events.



Visual vs collision meshes at t=8s during idle-stripped key-action replay

Simulation timing

Teleop and replay run at `fps=30`, so the Genesis control timestep is `dt=1/fps` (about 33 ms per step). Within each control step the rigid solver advances with `substeps=8`, refining contacts and constraints between recorded key-actions.

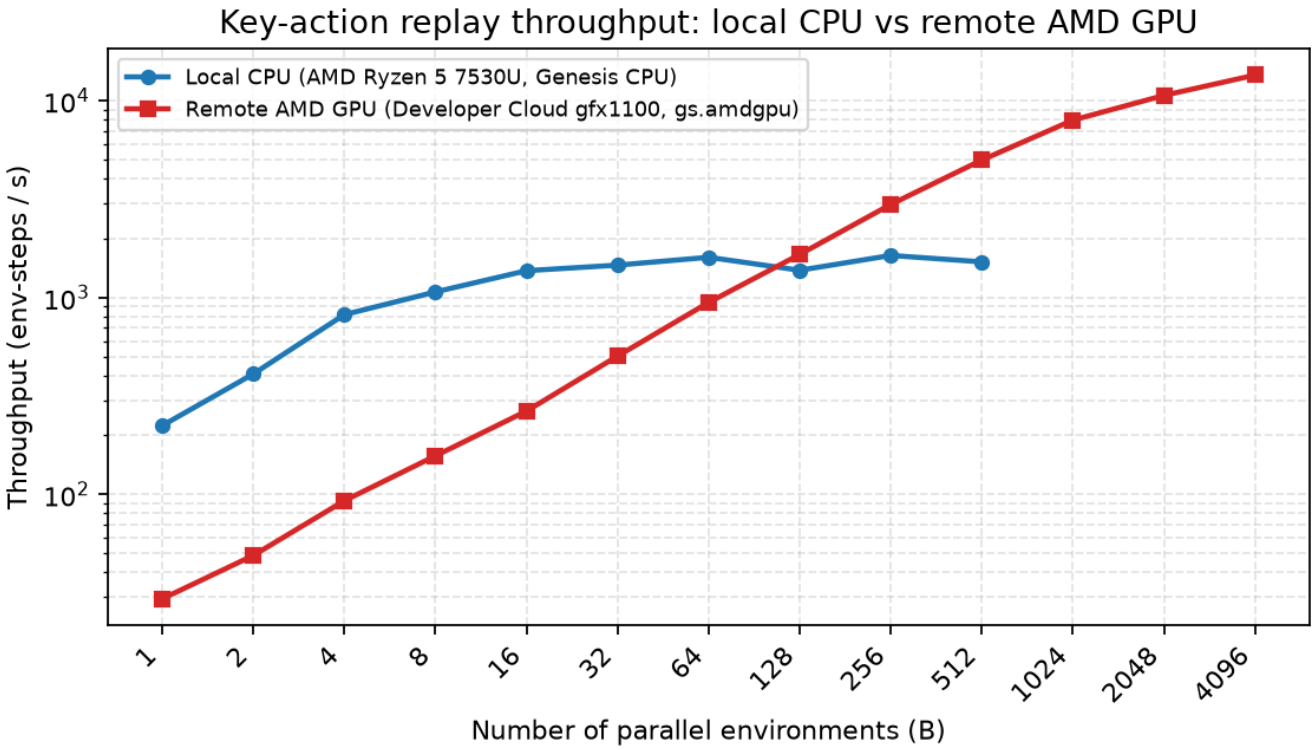
Noslip iterations for stable grasps

Even with good meshes, rigid-body solvers can allow the grasped object to slip between the fingers under gravity and inertial loads. Genesis exposes `noslip_iterations` on the rigid options; ArmForge sets this equal to `substeps` (`noslip_iterations=8`) during teleop and replay so the constraint solver spends 1 noslip pass per substep suppressing relative tangential motion at contacts. Without those iterations the cube tends to ooze out of the gripper during lift or transport, poisoning both human demos and policy learning. CoACD and noslip therefore work together: CoACD places contacts where the real geometry is, and noslip keeps the contact manifold stuck during the pick.

Multi-environment simulation: CPU cores versus GPU shaders

Once demos exist, scaling experience collection becomes the bottleneck. Genesis can step many environments in one batched call. On a CPU that parallelism is bounded by a small number of general-purpose cores. A modern laptop or workstation might expose on the order of tens of logical cores; beyond roughly that width, additional environments contend for the same cores and throughput plateaus. Our local CPU key-action replay already shows this: env-steps per second climb from a few hundred at `B=2` toward about 1600 by `B=64`-`B=256`, then flatten near 1500-1600 even at `B=512`. A GPU, by contrast, exposes thousands of shader (compute) units designed for the same kernel across many data-parallel instances. Rigid-body steps, batched kinematics, and vectorized policy inference map naturally onto that SIMD style

of work. Multi-environment simulation is therefore particularly relevant for GPU training compared with CPU training: the GPU's advantage is not merely clock speed, but width-many more concurrent lanes than CPU cores-so large environment batches keep the device busy while a CPU saturates early.



Key-action replay throughput: local CPU vs remote AMD GPU

The chart compares local `gs.cpu` throughput on an AMD Ryzen 5 7530U (12 threads) against remote AMD GPU throughput on AMD Developer Cloud (`gfx1100`, `gs.amdgpu`) for the same key-action episode replay workload. At small `B` the CPU is faster because the GPU pays launch and host-device overhead. From about `B=128` upward the remote AMD GPU overtakes and keeps climbing (about 5000 env-steps/s at `B=512`, about 13500 at `B=4096`) while the CPU remains flat-exactly the cores-versus-shaders gap that matters for RL data collection.

GPUs for reinforcement learning

That width makes GPUs a strong fit for reinforcement learning algorithms such as PPO, SAC, and DQN. These methods consume large volumes of transitions, benefit from decorrelated rollouts across parallel envs, and interleave simulation with neural network updates that already run on the accelerator. ArmForge trains privileged teachers with Stable-Baselines3 on top of Genesis vectorized envs (continuous control for PPO/SAC and discrete key-action DQN), targeting hundreds of parallel environments per worker. The same AMD ROCm path used for teleop (`gs.amdgpu`) is the intended backend for collecting those rollouts on Radeon hardware, with Docker packaging for Developer Cloud and local ROCm setups.

RL training can therefore run on an AMD GPU as long as the policy does not require visual

input. Privileged teachers observe cube/disk/robot state with cameras disabled (`enable_cameras=False`), so the hot path stays physics, kinematics, and network updates on `gs.amdgpu`. Madrona's batched renderer remains CUDA-only; vision-based students still need rasterizer cameras or a CUDA path for image observations, but state-based PPO, SAC, and DQN do not. When visual observations are required and neither Madrona nor a suitable AMD rasterizer path is available, the workaround is to run on CPU only (`gs.cpu`): simulation and (slower) rasterizer cameras stay correct, at the cost of the multi-env scaling shown above.

How GPU scaling speeds up training

Empirically, GPU throughput continues to scale with environment count long after CPU scaling stalls. On an AMD Developer Cloud `gfx1100` node, key-action multi-env replay rises from tens of env-steps per second at `B=1` to thousands at hundreds of environments and into the 10000+ range near `B=4096`. That curve matters for training time: wall-clock progress in PPO, SAC, or DQN is largely proportional to how many environment steps you can gather per second. When each extra environment still adds useful parallel work on the GPU's shaders, the same hyperparameter schedule finishes sooner; when a CPU has already exhausted its cores, buying more environments mostly adds scheduling overhead. Multi-env Genesis on AMD GPUs therefore shortens the path from teleoperated contact skills to learned policies by pairing human teaching with scalable, ROCm-native data collection.